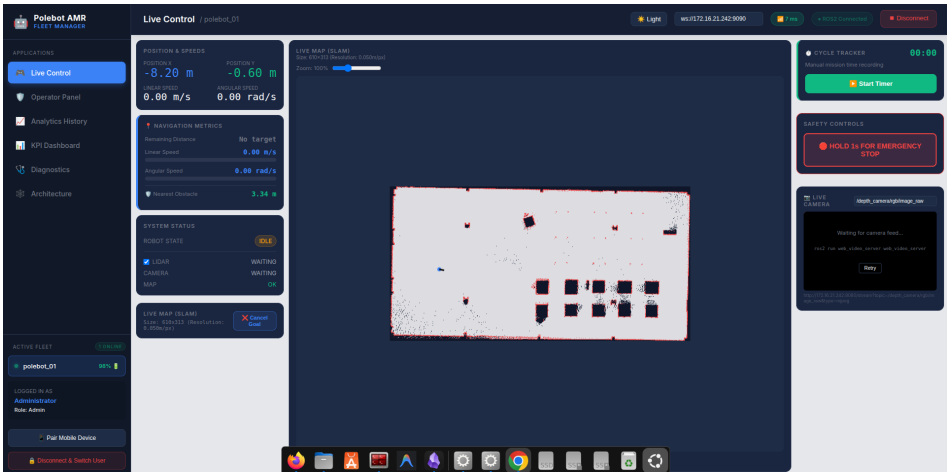
Technical Documentation

Web-Based ROS Supervision Platform for the Polebot Autonomous Mobile Robot

Edouard PERDRIX

Supervised by M. Wahyu Adhie CANDRA

Robotik DCS & SCADA Laboratory — Polman Bandung, Indonesia



Contents

1 Introduction

1.1 Purpose of this document

This document provides the technical documentation for the web-based supervision platform developed during the internship at the Robotik DCS & SCADA Laboratory of Polman Bandung. It is intended for developers and engineers who need to understand, maintain, or extend the system.

The platform was built to address a real operational need: the Polebot AMR, an autonomous mobile robot developed by the laboratory, lacked a supervision interface capable of monitoring its telemetry in real time, storing historical data, and generating performance indicators. This documentation covers the complete technical stack, from the ROS 2 communication layer to the web application, including the data storage pipeline and the API endpoints.

It is important to note that the interface was fully developed and validated in the Gazebo simulation environment, where all features - real-time SLAM map rendering, teleoperation, InfluxDB data ingestion, KPI computation, and PDF/CSV export - were tested and confirmed to work as expected.

Physical integration was attempted on two legacy versions of the Polebot AMR. Both attempts were unsuccessful in achieving full supervision functionality for the following reasons :

- **Legacy version 1 (Arduino Uno, mains-powered):** the motor control chain was validated — the interface successfully sent commands via /cmd_vel and the robot moved. SLAM mapping was partially achieved: a partial map was successfully generated and rendered in the dashboard, confirming that the full chain (LiDAR -> SLAM Toolbox -> /map topic -> Web Worker rendering) works end-to-end. However, a systematic timestamp desynchronization between the LiDAR clock and the ROS 2 system clock intermittently degraded the mapping quality. Despite multiple corrective measures (timestamp relay node, increased transform timeout, TF broadcaster at 50 Hz), SLAM Toolbox continued to intermittently report Failed to compute odom pose, preventing a complete and reliable map from being obtained. Additionally, this version runs on mains power with no battery, so no battery telemetry was available.

- **Legacy version 2 (LattePanda):** SSH access could not be established due to unknown credentials, and no HDMI-compatible screen was available in the laboratory to retrieve the IP address and configure the connection. Physical integration on this version therefore could not be attempted.

A third and most recent version of the robot was identified during the internship. It features a significantly more complete electronics stack: dedicated motor drivers, an integrated power management system, and an onboard battery.

However, this version could not be tested within the scope of this internship. Without available technical documentation on its software architecture, it was impossible to reconstruct the complete ROS 2 environment required to conduct relevant tests — topic identification, rosbridge configuration, motor driver verification. Furthermore, as the internship ends this week, integration of this version could not be completed within the allotted time.

It nonetheless represents the natural next target for integration work by future laboratory teams.

This document is structured as follows: the system architecture section provides a high-level overview of the platform, followed by a detailed description of the software architecture, the data schema, the API endpoints, the security mechanisms, and finally the deployment guide.

2 System Architecture

2.1 Overview

The platform follows a three-layer architecture that clearly separates the robotics layer, the transport layer, and the web application layer. This separation ensures that each layer can be updated or replaced independently without affecting the other. Figure 1 provides a visual overview of the complete communication architecture.

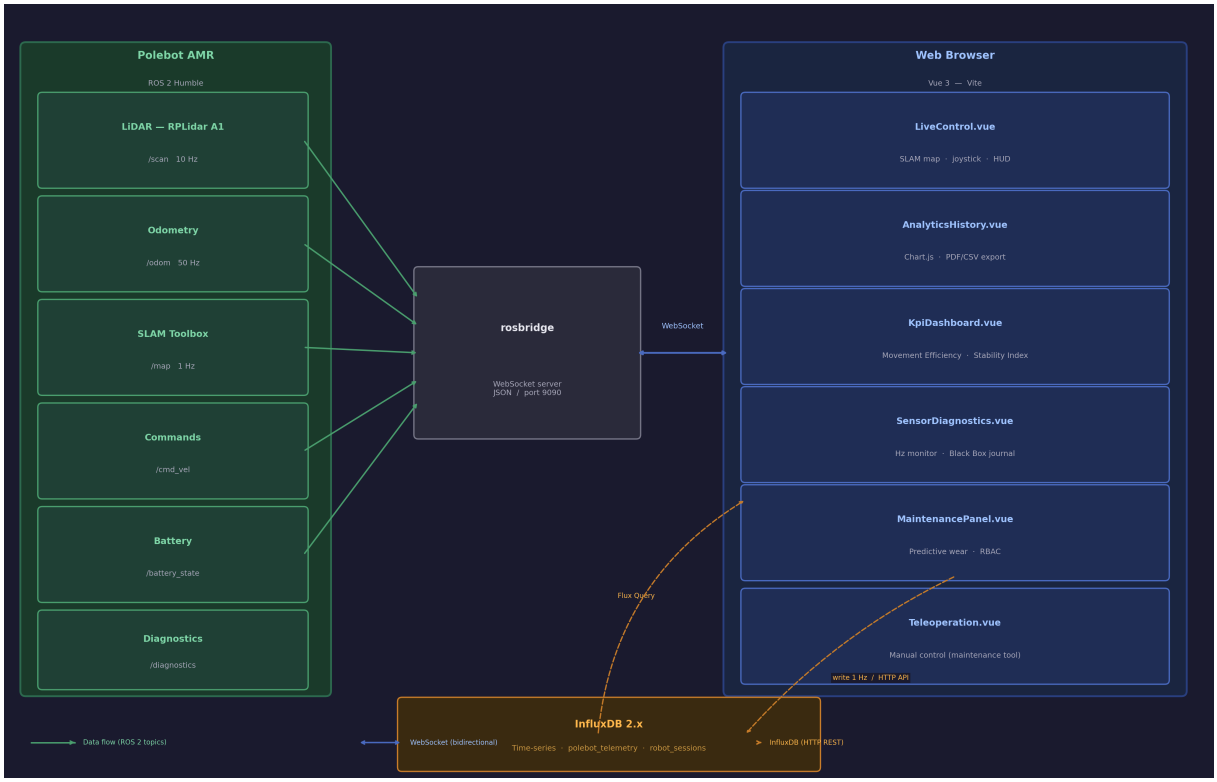


Figure 1: System communication architecture

2.2 Three-Layer Description

Robotics Layer (ROS 2) — This layer runs on the robot's embedded computer (Intel NUC or LattePanda depending on the robot version) under Ubuntu 22.04 and ROS 2 Humble. It is responsible for sensor data acquisition (RPLidar A1 on `/scan`), odometry computation (`/odom`), SLAM map generation (`/map`), and motor control via `/cmd_vel`. The `rosbridge_suite` package acts as the gateway between this layer and the outside world.

Transport Layer (Middleware) — This layer handles all communication between the robot and the web application. It consists of two independent channels:

- A **WebSocket** connection (port 9090) managed by `rosbridge_suite`, enabling bidirectional real-time exchange of ROS 2 messages in JSON format.
- An **HTTP REST** connection (port 8086) to InfluxDB 2.x, used for writing telemetry at 1 Hz and retrieving historical data via Flux queries.

Web Application Layer (Browser) - This layer is a Vue 3 Single-Page Application running entirely in the browser. It connects to both transport channels simultaneously - the WebSocket for real-time data and the InfluxDB HTTP API for historical analytics.

2.3 Communication Flow

As illustrated in Figure 1, the data flows through the three layers as follows:

1. The robot's ROS 2 nodes publish sensor data on topics (`/scan`, `/odom`, `/map`).
2. `rosbridge_suite` translates these messages into JSON and exposes them over a WebSocket on port 9090.
3. The Vue 3 application subscribes to topics via `roslibjs` and renders data in real time.
4. In parallel, `App.vue` writes telemetry to InfluxDB every second via HTTP POST.
5. Historical data is retrieved via Flux queries and displayed in Chart.js graphs.

2.4 Technology Stack

Table 1 summarizes the technologies used and the justification for each choice.

Technology	Version	Justification
Vue.js 3	3.x	Progressive framework, lowest learning curve, open-source
Vite	5.x	Fast build tool with Hot Module Replacement
roslibjs	latest	Only official JavaScript library for ROS 2 via WebSocket
rosbridge_suite	ROS 2 Humble	Standard WebSocket bridge for ROS 2
InfluxDB	2.x	Time-series database optimized for high-frequency sensor data
Chart.js	4.x	Lightweight visualization library for time-series charts
jsPDF	latest	Client-side PDF generation without server dependency
Web Audio API	Browser native	Real-time alarm synthesis without external audio files

Table 1: Technology stack and justification

3 Software Architecture

3.1 Project Structure

The application follows a modular architecture based on Vue 3's Composition API. The project is organized into two main directories : `components/`, which contains the visual pages of the dashboard, and `composables/`, which contains the shared business logic.

```
polebot_analytics/dashboard/
|-- dist/                          # Production build (Vite output)
|-- src/
|   |-- App.vue                    # Root component, InfluxDB write loop
|   |-- components/
|   |   |-- LiveControl.vue        # Real-time map, HUD, LiDAR overlay
|   |   |-- OperatorPanel.vue      # Operator dashboard, fleet status
|   |   |-- AnalyticsHistory.vue   # Historical charts, PDF/CSV export
|   |   |-- KpiDashboard.vue       # Movement Efficiency, Stability Index
|   |   |-- SensorDiagnostics.vue  # Topic Hz monitor, Black Box journal
|   |   |-- MaintenancePanel.vue   # Predictive wear estimator
|   |   |-- RosGraph.vue           # SVG node/topic graph
|   |   |-- Teleoperation.vue      # Manual control (maintenance tool)
|   |-- composables/
|       |-- useRos.js              # ROS 2 singleton + demo simulator
|       |-- useBattery.js          # Battery simulation / real reading
|       |-- useBlackBox.js         # Persistent incident journal
|       |-- useAudio.js            # Web Audio alarm synthesizer
|       |-- useAuth.js             # RBAC (Admin / Operator)
|-- vite.config.js
|-- package.json
```

Listing 1: Project directory structure

3.2 Composables

In Vue 3, a composable is a function that encapsulates reactive state and logic, and can be shared across multiple components as a singleton. This pattern avoids prop drilling and ensures that all components share the same state without duplication.

`useRos.js` — **ROS 2 Connection Manager.** This is the central composable of the application. It manages the WebSocket connection to `rosbridge_suite`, subscribes to ROS 2 topics (`/odom`, `/scan`, `/cmd_vel` and `/goal_pose`). It also implements the heartbeat mechanism (ping every 500 ms), automatic reconnection after three consecutive failures, and the low-bandwidth mode triggered when RTT exceeds 150 ms over three consecutive measurements. The global E-STOP state (`isEStopActive`) is managed here and shared across all components, ensuring that an emergency stop triggered from any page immediately blocks all movement commands.

`useBattery.js` — **Battery Management.** This composable handles the battery level displayed in the dashboard. In simulation mode, it discharges the battery at 0.0093% per second when the robot is moving, which corresponds to approximately 3 hours of autonomy. This rate was chosen to be realistic for an industrial AMR. When the physical robot with a battery sensor is available, the composable can switch to real mode by subscribing to the `/battery_status` topic and disabling the simulation.

`useBlackBox.js` — **Incident Journal.** This composable implements the black box functionality required by the internship proposal. It maintains a persistent journal of the 100 most recent incidents, stored in `localStorage` so that it survives page reloads. Seven events are automatically recorded: connection drop, connection established, E-STOP active, E-STOP released, collision risk, low battery, and log cleared.

`useAudio.js` — **Alarm Synthesizer.** Rather than relying on external audio files, this composable generates all alarms in real time using the browser's Web Audio API. This ensures near-zero latency and eliminates any dependency on file loading. Two alarms are implemented: the E-STOP alarm (two sawtooth oscillators at 440 Hz and 380 Hz creating a 60 Hz acoustic beat) and the collision alert (a triangular oscillator ramping from 600 Hz to 1200 Hz in 0.15 seconds).

`useAuth.js` — **Role-based Access Control.** This composable manages user authentication and access control. Two roles are defined: Administrator (`polebot01`), who has access to all tabs, and Operator (`polebot02`), who is restricted to the Live Control and Operator Panel tabs. The session token is stored in `sessionStorage`, which is automatically cleared when the browser tab is closed, preventing an operator from remaining inadvertently logged in on a shared workstation. You can find the username and password programmed in the file users.js.

3.3 Components

The dashboard is organized into eight Vue components, each corresponding to a specific operational need. A key architectural decision was made during development: the teleoperation controls were initially integrated into the main `LiveControl.vue` page. Upon re-reading the internship proposal, which specifies that the supervision interface should remain read-only or limited control, the joystick and movement controls were moved to a dedicated `Teleoperation.vue` component accessible via a separate maintenance tab. This ensures a clear separation between operational supervision and manual control.

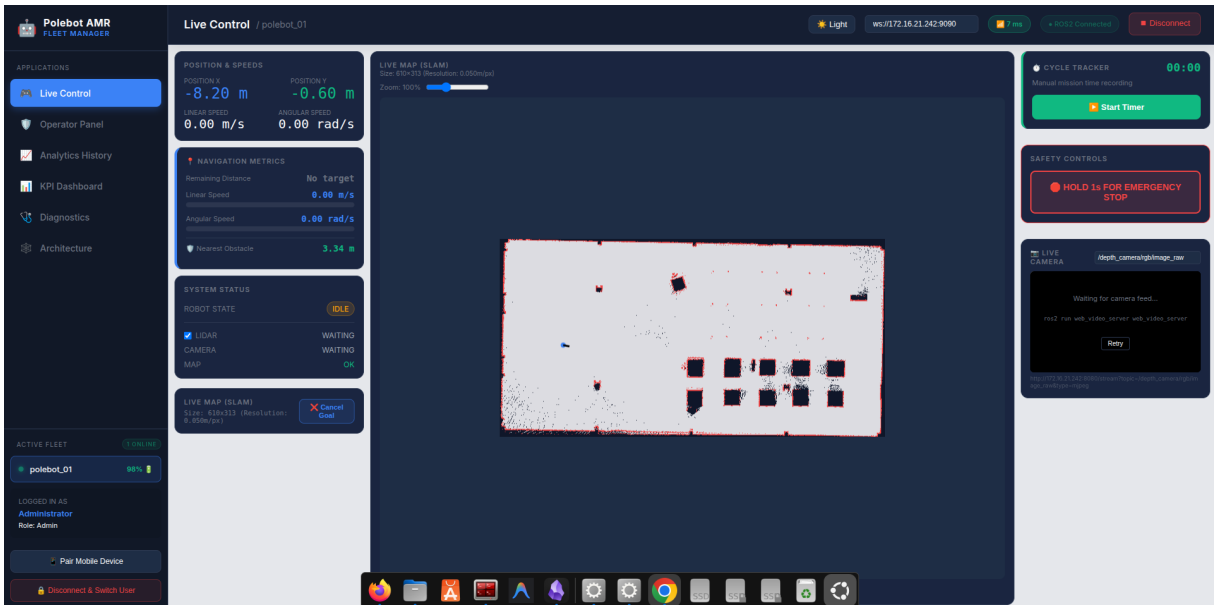


Figure 2: Polebot AMR supervision dashboard — Live Control page

3.4 SLAM Map Rendering Pipeline

The `/map` topic publishes an `OccupancyGrid` message that can contain up to 1000×1000 cells, representing up to 1 MB of data per second transmitted over the WebSocket. Rendering this matrix pixel by pixel in Vue 3's main thread caused complete interface freezing (*UI freeze*).

The solution adopted relies on a **Web Worker**: a JavaScript program running in a separate browser thread, without access to the DOM or the graphical interface. The main thread immediately delegates the raw `/map` data to the Web Worker, which decodes and renders it pixel by pixel onto an `OffscreenCanvas` — an invisible drawing surface identical to the standard HTML `<canvas>` but usable outside the main thread. Once rendering is complete, the image is transferred back to the main thread by memory copy and displayed on screen. The robot position is then overlaid as a blue dot computed mathematically from its odometry position.

3.5 Demo Simulator

Since physical access to the robot is not always available, I implemented a built-in demo simulator that allows the dashboard to be demonstrated and tested without any robot

connection. When demo mode is activated, the WebSocket connection is bypassed and a 10 Hz loop generates synthetic telemetry data:

- Circular trajectory: $X(t) = 2.5\cos(0.15t)$, $Y(t) = 2.5\sin(0.15t)$
- Heading: $\theta(t) = (0.15t \bmod 2\pi) - \pi$
- Simulated sensor frequencies: odometry at 50 Hz, LiDAR at 10 Hz, map at 1 Hz
- Simulated network latency: random oscillation between 12 and 17 ms

All dashboard features — KPI computation, InfluxDB ingestion, alarm triggering, black box logging — function identically in demo mode and in real robot mode.

4 Data Schema

4.1 Overview

The platform uses two data sources with different roles. ROS 2 topics provide real-time data streamed directly to the web interface via WebSocket. InfluxDB stores historical telemetry for analytics and reporting. Unlike relational databases (PostgreSQL, MySQL), InfluxDB is a time-series database — there are no tables, no primary keys, and no foreign keys. Data is organized around **measurements**, **tags**, and **fields**.

- **Measurement** — equivalent to a table name (e.g. `telemetry`, `session`).
- **Tags** — indexed metadata used for filtering (e.g. `robot_id`). Tags are strings and are optimized for queries.
- **Fields** — the actual measured values (e.g.`linear_speed`, `battery`). Fields can be floats, integers, booleans, or strings.
- **Timestamp** — every data point is automatically associated with a nanosecond-precision Unix timestamp.

4.2 InfluxDB Data Model

The platform uses two buckets, both with a retention policy of 30 days.

Bucket `polebot_telemetry`. This bucket stores the robot's operational telemetry, written at **1 Hz** by `App.vue` via an asynchronous interval.

Field / Tag	Kind	Type	Unit	Description
`robot_id`	Tag	string	—	Robot identifier. Used to filter multi-robot queries.
`linear_speed`	Field	float	m/s	Linear speed from `/odom`
`angular_speed`	Field	float	rad/s	Angular speed from `/odom`
`pos_x`	Field	float	m	X position from `/odom`
`pos_y`	Field	float	m	Y position from `/odom`
`yaw`	Field	float	rad	Heading from `/tf`
`battery`	Field	float	%	Battery level (simulated or real sensor)
`obstacle_dist`	Field	float	m	Minimum obstacle distance from `/scan`
`estop_active`	Field	boolean	—	Emergency stop state (HMI event)

Bucket `robot_sessions`. This bucket stores session metadata, written at session start and end by `App.vue`. It allows estimation of the MTBF (*Mean Time Between Failures*) over the long term.

Field / Tag	Kind	Type	Description
`robot_id`	Tag	string	Robot identifier
`session_id`	Field	string	Unique session identifier
`start_time`	Field	integer	Session start Unix timestamp (s)
`end_time`	Field	integer	Session end Unix timestamp (s)
`duration_s`	Field	float	Total session duration (s)

4.3 Line Protocol

InfluxDB uses its own text format called **Line Protocol** for writing data points. Each line follows this structure:

```
<measurement>,<tag>=<value> <field>=<value> <timestamp>
```

Example — telemetry point:

```
telemetry,robot_id=polebot_01
  linear_speed=0.35,angular_speed=0.0,
  pos_x=1.23,pos_y=-0.45,yaw=0.78,
  battery=87.3,obstacle_dist=2.1,
  estop_active=false 1783049638000000000
```

Example — session record:

```
session,robot_id=polebot_01
  session_id="sess_001",
  start_time=1783049600,
  end_time=1783053200,
  duration_s=3600.0 1783053200000000000
```

4.4 ROS 2 Topic Schema

The following table lists all ROS 2 topics used by the platform, their message types, publication frequencies, and the key fields extracted for display or storage.

Topic	Message type	Hz	Key fields used
`/odom`	`nav_msgs/Odometry`	50	`twist.linear.x`, `twist.angular.z`
`/robot_pose`	`geometry_msgs/PoseStamped`	—	`pose.position.x/y`, orientation
`/scan`	`sensor_msgs/LaserScan`	10	`ranges[]`, `range_min/max`
`/map`	`nav_msgs/OccupancyGrid`	1	`data[]`, `info.width/height`
`/diagnostics`	`diagnostic_msgs/DiagnosticArray`	—	`status[].level`
`/cmd_vel`	`geometry_msgs/Twist`	—	`linear.x`, `angular.z`
`/goal_pose`	`geometry_msgs/PoseStamped`	—	`pose.position.x/y`

4.5 Data Flow Diagram

Figure 3 illustrates how data flows from the ROS 2 topics through the transport layer to the web interface and InfluxDB.

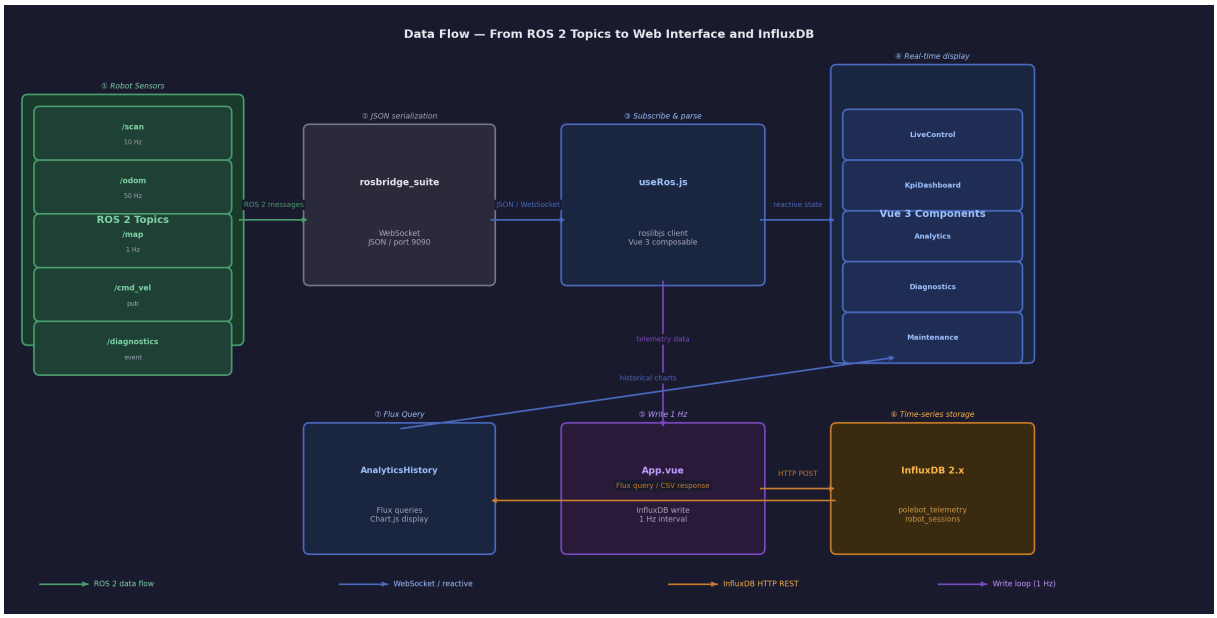


Figure 3: Data flow from ROS 2 to the web interface and InfluxDB

5 API & Endpoints

5.1 Overview

The platform interacts with two external APIs: the **InfluxDB 2.x REST API** for time-series data storage and retrieval, and the **rosbridge WebSocket API** for real-time ROS2 communication. Both APIs are third-party interfaces that the platform consumes — no custom backend API was developed as part of this project.

5.2 InfluxDB REST API

Base URL: `http://localhost:8086`

Authentication: all requests require an API token passed in the HTTP header:

```
Authorization: Token <api_token>
```

Method	Endpoint	Description
POST	`/api/v2/write?org={org}&bucket=polebot_telemetry&precision=ns`	Write a telemetry data point. Body: Line Protocol.
POST	`/api/v2/write?org={org}&bucket=robot_sessions&precision=ns`	Write session metadata. Body: Line Protocol.
POST	`/api/v2/query?org={org}`	Execute a Flux query. Body: `application/vnd.flux`. Returns CSV.
GET	`/health`	Check InfluxDB availability. Returns `200 OK` if healthy.

Example — write request (HTTP POST):

```
POST /api/v2/write?org=polebot&bucket=polebot_telemetry&precision=ns
Authorization: Token <api_token>
Content-Type: text/plain; charset=utf-8

telemetry,robot_id=polebot_01
  linear_speed=0.35,angular_speed=0.0,
  pos_x=1.23,pos_y=-0.45,battery=87.3,
  estop_active=false 1783049638000000000
```

Example — Flux query (HTTP POST):

```
POST /api/v2/query?org=polebot
Authorization: Token <api_token>
Content-Type: application/vnd.flux

from(bucket: "polebot_telemetry")
  |> range(start: -10m)
  |> filter(fn: (r) => r.robot_id == "polebot_01")
  |> filter(fn: (r) => r._field == "linear_speed")
  |> aggregateWindow(every: 5s, fn: mean)
```

Error responses:

HTTP Code	Description
`200 OK`	Request successful.
`204 No Content`	Write successful (no response body).
`401 Unauthorized`	Invalid or missing API token.
`404 Not Found`	Bucket or organization not found.
`422 Unprocessable`	Malformed Line Protocol data.
`500 Server Error`	InfluxDB internal error.

5.3 Rosbridge WebSocket API

URL: `ws://{robot_ip}:9090`

The rosbridge WebSocket API uses JSON messages conforming to the `rosbridge_protocol` specification. Each message contains an `op` field that defines the operation type.

Operation	`op` field	Description
Subscribe	`"subscribe"`	Subscribe to a ROS 2 topic. Messages are received asynchronously via callback.
Publish	`"publish"`	Publish a message to a ROS 2 topic.
Advertise	`"advertise"`	Declare intent to publish on a topic before sending messages.
Call service	`"call_service"`	Call a ROS 2 service and receive the response synchronously.

Example — subscribe to `/odom`:

```
{
  "op": "subscribe",
  "topic": "/odom",
  "type": "nav_msgs/msg/Odometry"
}
```

Example — publish to `/cmd_vel`:

```
{
  "op": "publish",
  "topic": "/cmd_vel",
  "msg": {
    "linear":  { "x": 0.3, "y": 0.0, "z": 0.0 },
    "angular": { "x": 0.0, "y": 0.0, "z": 0.0 }
  }
}
```

Error handling:

The platform implements two mechanisms to handle rosbridge connection failures:

- **Heartbeat:** a ping message is sent every 500 ms. If three consecutive responses are missing, the connection is declared lost and automatic reconnection is triggered.
- **Low bandwidth mode:** if the round-trip time (RTT) exceeds 150 ms over three consecutive measurements, the platform automatically reduces the `/map` rendering frequency from 1 Hz to 0.2 Hz to preserve bandwidth for safety-critical commands.

6 Security & Resilience

6.1 Authentication & Access Control (RBAC)

The platform implements a Role-Based Access Control system directly in the front-end. Two roles are defined.

Role	Credentials	Accessible tabs
Administrator	`polebot01`	All tabs (Live Control, Operator Panel, Analytics, KPI, Diagnostics, Architecture, Maintenance)
Operator	`polebot02`	Live Control and Operator Panel only

The session token is stored in `sessionStorage`, which is automatically cleared when the browser tab is closed, preventing an operator from remaining inadvertently logged in on a shared workstation.

Known Limitation — Hardcoded Credentials

In the current implementation, credentials are hardcoded directly in the front-end code (`user.js`). This is acceptable for a laboratory prototype but would need to be replaced by a proper backend authentication service (e.g. JWT tokens, OAuth2) before any production deployment. No HTTPS is configured either, as the platform runs on a local network.

6.2 Network Resilience

Heartbeat mechanism. To detect silent WebSocket disconnections — where the browser maintains the socket in an "open" state after a sudden Wi-Fi cut — I implemented a heartbeat system. A ping message is sent to rosbridge every 500 ms. If three consecutive responses fail to arrive within the expected delay, the connection is declared lost and an automatic reconnection attempt is triggered. The incident is simultaneously logged in the black box as a **Critical** event.

Low Bandwidth mode. When the round-trip time (RTT) exceeds 150 ms over three consecutive measurements, the platform automatically switches to a degraded mode:

- The `/map` rendering frequency is reduced from 1 Hz to 0.2 Hz, freeing bandwidth for safety-critical commands.
- Secondary LiDAR point tracing is suspended.
- A warning badge is displayed in the interface header

This mode was triggered during physical tests on the legacy robot, where the school Wi-Fi network produced RTT values between 158 ms and 332 ms.

Global E-STOP state. The emergency stop state (`isEStopActive`) is managed as a global reactive variable in `useRos.js`, shared across all components. When the E-STOP is triggered from any page, all movement commands are immediately blocked regardless of the active tab.

6.3 Error Handling

WebSocket errors. All WebSocket errors are caught by the heartbeat mechanism and trigger automatic reconnection. The incident is logged in the black box with a **Critical** severity level.

Black Box. The Black box (`useBlackBox.js`) acts as the main error reporting system for operational events. it persists across page reloads via `localStorage` and retains the 100 most recent incidents, providing a complete audit trail of all critical events during a session.

6.4 Known Limitations

The following security limitations are acknowledged and should be addressed before any production deployment:

- **Hardcoded credentials:** the Admin and Operator passwords are stored in plain text in `useAuth.js`. A proper authentication backend with hashed passwords and JWT tokens should be implemented.
- **No HTTPS:** all communications (WebSocket and InfluxDB HTTP) are unencrypted. On a local laboratory network this is acceptable, but not suitable for remote access.
- **No InfluxDB write confirmation:** failed writes are silently ignored in the current implementation.
- **Front-end only RBAC:** the access control is enforced only in the browser. A determined user could bypass it by directly calling the rosbridge or InfluxDB API.

7 Deployment Guide

7.1 Prerequisites

The following software must be installed before deploying the platform:

Software	Version	Notes
Node.js	≥ 18	Required to run the Vue 3 development server
npm	≥ 9	Installed with Node.js
InfluxDB	2.x	Must be running on `localhost:8086`
ROS 2	Humble	On the robot's embedded computer
rosbridge_suite	ROS 2 Humble	`sudo apt install ros-humble-rosbridge-suite`

7.2 Source Code

The complete source code of the platform is available on GitHub:

https://github.com/edp1806/Polebot-AMR

The repository contains the following directories:

- `polebot_analytics/dashboard/` — Vue 3 web application

- `cmd_odom_serial.py` — ROS 2 motor control and odometry bridge script
- `fix_scan_timestamp.py` — LiDAR timestamp synchronization relay node
- `Polaris_Polman/slam_minimal.yaml` — SLAM Toolbox configuration file

7.3 Installation

Step 1 — Clone the project and install dependencies:

```
    cd polebot_analytics/dashboard
    npm install
```

Step 2 — Configure InfluxDB:

Create a bucket named `polebot_telemetry` and a second bucket named `robot_sessions` in InfluxDB. Generate an API token with read/write access to both buckets.

Then configure CORS in the InfluxDB configuration file to allow requests from the Vue 3 development server:

```
# In influxdb.conf
[http]
  cors-enabled-origins = ["http://localhost:5173"]
```

Step 3 — Start the development server:

```
npm run dev
# Interface available at http://localhost:5173
```

Step 4 — Build for production:

```
npm run build
# Production files generated in /dist
```

7.4 Robot Launch Sequence

Connect to the robot's embedded computer via SSH and launch the following services in order. Each service must be fully started before launching the next one.

Terminal 1 — rosbridge (launch first, wait 5 seconds):

```
ssh polaris@<robot_ip>
export ROS_DOMAIN_ID=0
source /opt/ros/humble/setup.bash
ros2 launch rosbridge_server rosbridge_websocket_launch.xml
```

Terminal 2 — Motor control and odometry (wait 10 seconds):

```
ssh polaris@<robot_ip>
export ROS_DOMAIN_ID=0
source /opt/ros/humble/setup.bash
python3 ~/cmd_odom_serial.py
```

Terminal 3 — LiDAR (wait 5 seconds):

```
ssh polaris@<robot_ip>
source ~/Polaris_Polman/install/setup.bash
ros2 launch rplidar_ros rplidar_a1_launch.py use_ros_time:=true
```

Terminal 4 — Static TF transform (wait 5 seconds):

```
ssh polaris@<robot_ip>
source /opt/ros/humble/setup.bash
ros2 run tf2_ros static_transform_publisher \
  --frame-id base_link --child-frame-id laser
```

Terminal 5 — SLAM (launch last):

```
ssh polaris@<robot_ip>
source ~/Polaris_Polman/install/setup.bash
ros2 launch slam_toolbox online_async_launch.py \
  params_file:=~/Polaris_Polman/slam_minimal.yaml
```

7.5 Configuration

The following parameters can be adjusted to match your environment:

Parameter	Default value	Description
WebSocket URL	`ws://localhost:9090`	Replace with the robot's IP address
InfluxDB URL	`http://localhost:8086`	Change if InfluxDB runs on a different host
InfluxDB token	—	API token generated during InfluxDB setup
InfluxDB org	`polebot`	Organization name configured in InfluxDB
ROS_DOMAIN_ID	`0`	Must be identical on all terminals

7.6 Simulation Launch Sequence

The platform can be fully demonstrated without a physical robot using the Gazebo simulator. The following commands launch the complete simulation environment:

Terminal 1 — Gazebo simulation:

```
cd ~/Desktop/polebotamr
colcon build
source install/setup.bash
ros2 launch polebot_amr_bringup \
  polebot_amr_sim_nav_webgui.launch.py
```

Terminal 2 — InfluxDB database:

```
    influxd
    # If already running as a system service, skip this step
    #Check: systemctl status influxdb
```

Terminal 3 — Web interface:

```
cd polebot_analytics/dashboard
npm run dev
# Interface available at http://localhost:5173
```

Accessing the dashboard.

1. Open Chrome or Firefox and navigate to `http://localhost:5173`
2. Log in with one of the following credentials:
   - Administrator: `polebot01` and the password is `polebot@amr01`
   - Operator: `polebot02` and the password is `polebot@amr02` or `polebot03` and the password is `polebot@amr03`
   - Click `Connect` and enter the WebSocket URL:
     - Simulation: `ws://localhost:9090`
     - Physical Robot: find the IP address with `ip a` in robot terminate. `ws://IP`$_{address}$: 9090

Troubleshooting.

Problem	Solution
`npm: command not found`	Install Node.js: `sudo apt install nodejs npm`
Interface does not connect to robot	Verify rosbridge is running on the robot
Analytics charts empty	Verify InfluxDB is running
Blank page	Clear browser cache (Ctrl+Shift+R)

7.7 Known Issues — Physical Robot Data Retrieval

Physical integration was attempted on two legacy versions of the Polebot AMR. The following issues were encountered and remain unresolved at the time of writing.

7.8 Physical Robot Integration — Achieved Results

Physical integration was conducted on the **legacy version 1** of the Polebot AMR (Arduino Uno, mains-powered). The following features were successfully validated:

Feature	Status	Notes
rosbridge WebSocket connection	**OK**	`ws://172.16.22.226:9090`
RPLidar A1 `/scan`	**OK**	6.5 Hz, `/dev/ttyUSB0`
SLAM 2D map generation	**Partial**	slam_toolbox, real-time
TF tree	**OK**	map→odom→base_link→laser
Simulated odometry	**OK**	fake_odom_publisher
Dashboard live connection	**OK**	Canvas 2D, LiDAR point cloud
Teleoperation	**OK**	Joystick/keyboard → /cmd_vel
Physical motors	**PENDING**	serial_bridge not yet connected
Real odometry (encoders)	**PENDING**	fake_odom to be replaced
Autonomous navigation (Nav2)	**PENDING**	Map saving + waypoints

Physical robot launch sequence. The complete system is launched with a single command on the robot:

```
ssh polaris@<robot_ip>
cd ~/Polebot_Fix/polebot_amr_ws
source install/setup.bash
ros2 launch polebot_amr_bringup \
  polebot_amr_real_rplidar.launch.py
```

This launch file starts simultaneously: the LiDAR driver, SLAM Toolbox, the fake odometry publisher, the pose publisher, and the rosbridge WebSocket server on port 9090.

Legacy version 1 (Arduino Uno, mains-powered): Motor control was successfully validated on this version. A partial SLAM map was also successfully generated and displayed in the dashboard, demonstrating that the complete mapping pipeline is functional. However, despite multiple corrective measures (timestamp relay node, increased transform timeout, TF broadcaster at 50 Hz), SLAM Toolbox intermittently reported Failed to compute odom pose, which degraded map quality and prevented a complete, reliable map from being obtained. The root cause is likely a fundamental incompatibility between the Arduino Uno's serial output rate and the timing requirements of SLAM Toolbox.

Legacy version 2 (LattePanda): SSH access could not be established due to unknown credentials, and no HDMI-compatible screen was available in the laboratory. Physical integration on this version could not be attempted.

Recommended path forward:

- **New robot version (dedicated motor drivers):** A third and most recent version of the robot was identified during the internship. It features a significantly more complete electronics stack: dedicated motor drivers, an integrated power management system, and an onboard battery. However, this version could not be tested within the scope of this internship — without available technical documentation on its software architecture, it was impossible to reconstruct the complete ROS 2 environment required to conduct relevant tests. Furthermore, as the internship ends this week, physical integration of this version could not be completed within the allotted time. It represents the natural next target for integration work by future laboratory teams.
- **SLAM alternative:** if the timestamp issue persists, consider using `hector_slam` instead of SLAM Toolbox, as it does not require odometry and relies solely on LiDAR data.